



OSTİM TECHNICAL UNIVERSITY

FACULTY OF ENGINEERING

Department of Software Engineering

INTERNATIONAL CONGRESS FULL PAPER

**VI. International British Congress on Interdisciplinary Scientific Research &
Practices**

**Zaman Sayar: Design and Implementation of an Integrated
Calendar, Task, Note, and Responsibility Management
Application**

*Design and Implementation of an Integrated Calendar, Task, Note, and
Responsibility Management Application: Zaman Sayar*

Author	Samed Alp ARSLAN
Student ID	220205012
ORCID	0009-0001-8341-5142
E-mail	220205012@ostimteknik.edu.tr
Institution	OSTIM Technical University, Department of Software Engineering
Academic Advisor	Asst. Prof. Dr. Ramin ABBASZADI
Paper Type	VI. International British Congress English Full Paper
Acceptance Information	Ref: AC-06036 – Accepted for Oral Presentation and Proceedings
Acceptance Date	15/05/2026
Congress Date and Venue	18–21 June 2026, London / United Kingdom

London, United Kingdom, 2026

CONGRESS ACCEPTANCE, ACADEMIC DECLARATION, AND ETHICAL STATEMENT

This document is the English full-text academic and technical report version of **Zaman Sayar**, accepted under reference number **AC-06036** on **15/05/2026** for oral presentation and inclusion in the proceedings of VI. International British Congress on Interdisciplinary Scientific Research & Practices. The accepted English title stated in the acceptance letter is “Design and Implementation of an Integrated Calendar, Task, Note, and Responsibility Management Application: Zaman Sayar”.

The study presents a single web-based software system that integrates time planning, task tracking, note taking, RACI-based responsibility management, OCR-supported task extraction, GitHub integration, ZamanChat, and a machine-learning-based intelligent suggestion module. The text is structured as an international congress full paper rather than as an internal course submission. Accordingly, the title page, abstract, method, findings, discussion, conclusion, references, and appendices are organized to satisfy both conference full-paper expectations and technical traceability requirements.

No real personal data were used for the machine-learning experiments. The models were developed and evaluated on a synthetic and anonymized Turkish task dataset. Role labels were used instead of personal names, and all measurement outputs were preserved with fixed random seeds for reproducibility. Application components such as OCR, GitHub integration, ZamanChat, and user feedback are reported according to their actual implementation. Limitations such as synthetic data, the AI-assisted/simulated readability panel, weak unsupervised clustering results, and unencrypted storage of the GitHub personal access token in the current prototype are explicitly stated.

The development notes file `GELISTIRME_NOTLARI.md` is treated as an internal evidence and memory document. It is not copied into the paper text, but it supports numerical consistency and traceability behind the reported results.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my academic advisor, Asst. Prof. Dr. Ramin Abbaszadi, for his guidance, academic support, and valuable contributions throughout the development of this study. I also thank Dr. Cemil Baki Kiyak, Lecturer Asli Goncu, and Dr. Cagla Aksoy for their support and contributions to the improvement of the project.

I am deeply grateful to all the instructors who have shaped my engineering perspective and professional vision, especially Eray Yagdereli, Iclal Cetin Tas, Nooshin Nemati Tolakan, and Mustafa Sami Cucen. Their guidance has had a lasting influence on my learning process.

Finally, I thank my family, especially my mother Ayfer Arslan, my father Sukru Arslan, my brother Yasar Arslan, and our family friend Onur Sumer, for their constant support and encouragement.

This study is not only the product of a software development process, but also the outcome of a long learning journey shaped by valuable academic and personal support.

ABSTRACT

This study presents **Zaman Sayar**, a web-based system that integrates time planning, task tracking, note taking, role-based responsibility management, software-development traceability, and rapid access to operational information within a single application. The implemented system includes calendar management, task and subtask tracking, detailed task assignment, audio notes, RACI matrix support, dynamic tables, Excel/CSV data import, statistical dashboards, OCR-supported task extraction from images, GitHub integration, a read-only natural-language querying component, and a machine-learning-based intelligent task suggestion module. In this respect, the study is not merely an interface prototype; it is an integrated software engineering application with separated presentation, business logic, data access, and auxiliary service layers.

The system was developed on a layered architecture based on ASP.NET Core MVC, Razor, Entity Framework Core, and SQL Server. The OCR component uses Python, Tesseract, EasyOCR, and Pillow for image preprocessing, text extraction, label matching, and task-field generation. The ZamanChat component is a read-only assistant interface based on Node.js, Express, and Socket.IO; it inspects the database schema and answers user queries within controlled boundaries. The GitHub integration retrieves repositories, issues, pull requests, activity, releases, and calendar-synchronizable due dates through the GitHub REST API by using a user/team-scoped personal access token.

Beyond integration, the system includes a reproducible machine-learning subsystem that, on a synthetic and anonymized Turkish task corpus (1200 records, 5-fold cross-validation), classifies task type, category, and priority with macro-F1 scores of 0.91, 0.92, and 0.93 respectively; estimates task duration with a mean absolute error of 1.20 hours (MAPE 17.6%, $R^2 = 0.90$); yields a 44.3% reduction in simulated lateness under a budget-matched scenario; suggests RACI responsibility with 89.1% out-of-fold agreement; and produces extractive Turkish summaries that attain a readability score of 4.50/5 on a 20-rater AI-assisted/simulated Likert panel. The project is further supported by 55 automated unit tests.

The evaluation shows that Zaman Sayar combines fragmented digital work tools into a single operational panel, makes the relationship between tasks and time visible, improves responsibility traceability through RACI, and reduces manual data entry, data retrieval, and decision-support effort through OCR, ZamanChat, GitHub, and intelligent suggestion support layers. Since the readability panel is not a real human-subject study and the machine-learning results are based on synthetic data, these points are reported transparently as limitations. Future work should focus on real-world validation, end-to-end test coverage, role-based authorization, secure token storage, accessibility, and user-experience maturity before production-level use.

Keywords: ASP.NET Core MVC, task management, calendar application, RACI, OCR,

Entity Framework Core, SQL Server, ZamanChat, GitHub integration, machine learning, text classification, TF-IDF, TextRank summarization, reproducible evaluation.

SHORT ABSTRACT

This full paper presents **Zaman Sayar**, an integrated web-based time, task, note, responsibility, and decision-support application. The system combines calendar management, task assignment, notes and audio notes, RACI responsibility tracking, dynamic tables, spreadsheet import, statistics, OCR-supported task extraction, GitHub integration, read-only natural-language querying, and a machine-learning-based intelligent suggestion module within a single ASP.NET Core MVC architecture.

The application is implemented with ASP.NET Core MVC, Razor, Entity Framework Core, SQL Server, Python, Node.js, Socket.IO, and the GitHub REST API. The machine-learning subsystem, `zaman_ai`, classifies Turkish task text by type, category, and priority; estimates task duration and reminder timing; suggests RACI responsibility; and produces extractive summaries. On a synthetic and anonymized Turkish task dataset with 1200 records and 5-fold cross-validation, the best models reach macro-F1 scores of 0.91, 0.92, and 0.93 for type, category, and priority; 1.20-hour MAE for duration estimation; 44.3% simulated lateness reduction; and 89.1% RACI agreement. The study also reports an AI-assisted/simulated readability panel for summarization and states its limitations transparently.

The resulting system demonstrates that time planning, operational records, responsibility visibility, software-development traceability, and measured intelligent assistance can be engineered together in a single undergraduate-level but congress-presentable software system.

Keywords: integrated task management, ASP.NET Core MVC, OCR, RACI, GitHub integration, machine learning, Turkish task text, reproducible evaluation.

Contents

CONGRESS ACCEPTANCE, ACADEMIC DECLARATION, AND ETHICAL STATEMENT	i
ACKNOWLEDGEMENTS	ii
ABSTRACT	iii
SHORT ABSTRACT	v
CONTENTS	vi
LIST OF TABLES	x
LIST OF FIGURES	xi
SYMBOLS AND ABBREVIATIONS	xii
0.1 Abbreviations	xii
0.2 Symbols	xii
1 INTRODUCTION	1
1.1 Aim	1
1.2 Problem Context	1
1.3 Scope and Boundaries	2
1.4 Original Contributions	2
1.5 Organization of the Paper	3
2 CONCEPTUAL BACKGROUND	4
2.1 Task Management and Time Planning	4
2.2 Personal Information Management	4
2.3 The RACI Approach	5
2.4 OCR-Supported Information Extraction	5
2.5 Web-Based Software Architecture	6

2.6	Natural-Language Interface and Read-Only Boundary	6
2.7	Data Model, Structured Data, and Traceability	6
2.8	Machine Learning for Turkish Task Text Processing	7
3	RELATED WORK	9
3.1	Task- and Calendar-Oriented Systems	9
3.2	Personal Information Management and Context Continuity	9
3.3	Responsibility Management and RACI	9
3.4	Structured and Flexible Data Management	10
3.5	OCR-Supported Information Extraction	10
3.6	Natural-Language Interfaces and Safe Data Access	10
3.7	Web Architecture, Layered Design, and Maintainability	11
3.8	Machine Learning for Task Text Classification and Summarization	11
3.9	Design Implications from the Literature	12
3.10	Positioning of Zaman Sayar	12
4	MATERIALS AND METHOD	14
4.1	Development Approach	14
4.2	System Requirements	14
4.3	Software Architecture	15
4.4	Project Structure	16
4.5	Data Model	16
4.6	Module Design	18
4.7	OCR Workflow	18
4.8	ZamanChat Workflow and Security Boundary	19
4.9	GitHub Integration	19
4.10	Machine-Learning Subsystem (zaman_ai)	20
4.11	Dataset and Ethics	21
4.12	Feature Extraction and Models	21
4.13	Evaluation Protocol	22

4.14	Evaluation Indicators	22
5	FINDINGS AND DISCUSSION	24
5.1	Application Outputs	24
5.2	Functional Verification	24
5.3	A1: Task Type, Category, and Priority Classification	25
5.4	A2: Duration Estimation and Lateness Reduction	26
5.5	A3: Note Summarization	26
5.6	A4: RACI Responsibility Suggestion	27
5.7	EK-3: Task Text Clustering	28
5.8	A5: Prototype, Measurement Report, ER Schema, and Anonymous Data . .	28
5.9	Intelligent Task Suggestion Interface (Prototype)	28
5.10	Interface Results	30
5.11	Engineering Strengths	31
5.12	Limitations	31
5.13	Discussion	31
6	CONCLUSION AND FUTURE WORK	33
6.1	Conclusion	33
6.2	Future Work	33
	REFERENCES	35
A	Appendix A: Technical Component Inventory	38
B	Appendix B: Source Code Excerpts	40
B.1	Application Startup and Dependency Registration	40
B.2	Data Access Layer and Entity Relationships	40
B.3	Creating a Task Record from OCR Output	41
B.4	Running the Python OCR Script from the Web Application	42
B.5	OCR Engine Selection on the Python Side	43
B.6	Read-Only Socket.IO Flow for ZamanChat	43

B.7 Dynamic Schema Reading Approach	44
C Appendix C: Database Schema	45
CURRICULUM VITAE	48

List of Tables

1	Engineering objectives of the study	1
2	Conceptual mapping of Zaman Sayar	8
3	Design implications derived from the literature	12
4	Comparison of common tool approaches and Zaman Sayar	13
5	System requirements of Zaman Sayar	14
6	Main technologies used in the architecture	15
7	Controller responsibility map	16
8	Module-based design summary	18
9	ZamanChat decision flow	19
10	zaman_ai modules and research goals	21
11	Functional verification summary	25
12	A1 – Classification performance (1200 records, 5-fold cross-validation) . .	25
13	A2 – Duration estimation performance (5-fold cross-validation)	26
14	A2 – Budget-matched lateness reduction simulation	26
15	A3 – TextRank summary quality (automatic proxy readability, 1–5)	27
16	A3 – Readability Likert panel (N=20, AI-assisted/simulated; primary measure)	27
17	A4 – RACI suggestion performance and acceptance rate (5-fold cross-validation)	27
18	EK-3 – Task text clustering metrics	28
19	Summary of TUBITAK 2209-A target achievement	29
20	Technical summary of project components	38
21	Suggested quality-assurance checklist	39
22	Summary of database relationships	46
23	Main data entities and responsibilities	47

List of Figures

1	Conceptual relation between task management and time planning	4
2	Preserving context in personal information management	5
3	Meaning of RACI roles around a task	5
4	Conceptual flow of OCR-supported task extraction	5
5	Conceptual MVC architecture used in Zaman Sayar	6
6	Natural-language interface and read-only boundary in ZamanChat	6
7	From structured data model to traceability	7
8	Feature extraction and decision-support output generation from task text . .	7
9	Software architecture of Zaman Sayar	15
10	Summary of the Zaman Sayar data model	17
11	OCR-supported task extraction workflow	18
12	In-application GitHub integration: connected account profile, repository cards, language labels, and navigation tabs for repositories, dashboard, is- sues, pull requests, activity, and releases.	20
13	Zaman Sayar main panel, task list, and ZamanChat view	24
14	Intelligent Task Suggestion interface: generating task type/category/prior- ity (A1), estimated duration/reminder (A2), RACI responsibility (A4), and extractive summary (A3), together with human-in-the-loop feedback.	29
15	Calendar and statistics modules	30
16	RACI and dynamic table interfaces	30
17	OCR and ZamanChat support screens	30
18	Zaman Sayar database schema and main relationship groups	45

SYMBOLS AND ABBREVIATIONS

0.1 Abbreviations

API	Application Programming Interface
ARI	Adjusted Rand Index
ASP.NET	Active Server Pages .NET
CSV	Comma-Separated Values
CV	Cross-Validation
EF Core	Entity Framework Core
F1	Harmonic mean of precision and recall
HGB	HistGradientBoosting
JSON	JavaScript Object Notation
LSA	Latent Semantic Analysis
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MVC	Model-View-Controller
NMI	Normalized Mutual Information
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
PAT	Personal Access Token
RACI	Responsible, Accountable, Consulted, Informed
ROC-AUC	Receiver Operating Characteristic – Area Under Curve
SQL	Structured Query Language
TF-IDF	Term Frequency–Inverse Document Frequency
UI	User Interface
UX	User Experience

0.2 Symbols

T	Completion rate, defined as the ratio of completed tasks to all evaluated tasks.
-----------------------	--

$N_{completed}$	Number of completed tasks.
N_{total}	Total number of tasks evaluated in the system.
$Y(d)$	Workload value calculated for a specific day d .
$w(p_i)$	Weight coefficient corresponding to the priority of task i .
c_i	Binary completion variable for task i .
S_m	Selection or success score calculated for OCR engine m .
F_m	Number of task fields extracted from the OCR output.
E_m	Label-matching success in the OCR output.
Q_m	Readability level of the raw OCR text.
C_m	Confidence value produced by the OCR engine.
C_{RACI}	RACI role coverage ratio.

1 INTRODUCTION

1.1 Aim

The aim of this study is to design and implement a modular but single-panel time and task management application that can be used in individual and team-based workflows. The application goes beyond a conventional calendar or to-do list by addressing the engineering objectives summarized in Table 1.

Table 1: Engineering objectives of the study

Objective	Description
Integrated workflow	Provide calendar, task, note, table, RACI, statistics, and support modules in a single system.
Modular software architecture	Separate presentation, business logic, data access, and auxiliary services for maintainability.
Traceable data model	Store tasks, calendar records, notes, comments, tables, OCR logs, GitHub settings, and feedback records on a relational data backbone.
Auxiliary automation	Reduce manual workload through image-to-task extraction and read-only natural-language data querying.
Intelligent decision support	Generate task type, category, priority, duration, summary, and RACI responsibility suggestions from Turkish task text.
Academic defensibility	Present the system as an explainable engineering product with architectural decisions, data models, limitations, and measurable results.

1.2 Problem Context

In digital working environments, individuals and small teams often switch between calendars, to-do lists, note-taking tools, spreadsheets, assignment tools, and reporting dashboards. Although this fragmented model appears flexible, it may cause repeated data entry, context loss, unclear task ownership, and limited visibility over daily workload. When the date of a task is stored in a calendar, its explanation in a separate note, its owner in a communication channel, and its measurable status in a spreadsheet, software support may increase cognitive burden instead of reducing it.

Zaman Sayar was developed as an applied response to this problem. It is an integrated system centered on time planning and task management, combining notes, audio records, RACI-based responsibility sharing, dynamic tables, Excel/CSV import, statistical monitoring, OCR-based task extraction, GitHub integration, read-only natural-language querying, and machine-learning-based intelligent task suggestion within the same web application. The core motivation is not only to record daily work, but also to manage it together with time, responsibility, explanation, software-development traceability, and data context.

1.3 Scope and Boundaries

The scope of the study covers the design, implementation, and engineering evaluation of an ASP.NET Core MVC web application. The system includes calendar management, task and subtask management, detailed task assignment, notes and audio notes, RACI matrix support, dynamic tables, spreadsheet import, statistical screens, OCR-supported task extraction, GitHub integration, the read-only ZamanChat assistant, and the `zaman_ai` machine-learning subsystem. These modules are designed to work on the same database and application backbone.

Because the project is evaluated as a graduation project and an international congress full paper, some production-level issues remain open for future work. For example, comprehensive role-based authorization, end-to-end testing, centralized logging, distributed monitoring, and production-scale multi-user deployment are not the primary delivery targets. However, the machine-learning subsystem is supported by 55 unit tests, and the overall architecture provides a modular basis for future hardening.

1.4 Original Contributions

The contribution of the study does not lie in simply placing known modules side by side. Rather, it lies in designing these modules to work together around the problem of time, task, information, and responsibility management. The main contributions are as follows:

1. An integrated working panel combining calendar, task, note, RACI, dynamic table, file import, statistics, OCR, chatbot, GitHub, and intelligent suggestion components has been developed.
2. A layered software architecture has been built on ASP.NET Core MVC, Entity Framework Core, and SQL Server, with Python and Node.js auxiliary services connected to the core application through controlled service boundaries.
3. An OCR-supported task extraction pipeline has been designed by comparing Tesseract and EasyOCR outputs, using Turkish label synonyms, and converting raw text into structured task fields.
4. A controllable natural-language interface, ZamanChat, has been implemented to inspect the database schema, perform read-only SELECT-based queries, and reject write-oriented user intentions.
5. A reproducible and fixed-seed machine-learning subsystem, `zaman_ai`, has been designed and integrated without disrupting the existing architecture. It classifies Turkish

task text, estimates duration and reminder timing, suggests RACI responsibility, and generates extractive summaries.

6. The machine-learning goals have been evaluated on synthetic and anonymized data using 5-fold cross-validation; classification, regression, summarization, RACI recommendation, and clustering results are reported with tables and confidence information.
7. The previous testing gap has been partially addressed by implementing 55 automated unit tests for the `zaman_ai` subsystem.

1.5 Organization of the Paper

The second section explains the conceptual background of task management, personal information management, RACI, OCR, machine learning, and web-based software architecture. The third section reviews related work and positions Zaman Sayar among existing tool approaches. The fourth section presents system requirements, software architecture, data model, module design, OCR, ZamanChat, GitHub integration, and the `zaman_ai` workflow. The fifth section discusses application outputs, user-interface results, functional verification, machine-learning results, and limitations. The final section provides conclusions and future work.

2 CONCEPTUAL BACKGROUND

This section defines the main concepts used throughout the paper. Since Zaman Sayar brings together task management, calendar use, note taking, RACI, dynamic tables, OCR, natural-language querying, machine learning, and web architecture, these concepts should be clarified before the implementation is discussed.

2.1 Task Management and Time Planning

Task management is not merely the listing of things to do. An effective task-management system should represent what the work is, who owns it, when it is due, how important it is, and how its completion status can be monitored. Bellotti et al. emphasize that task managers should provide actionable information to users [1]. This perspective transforms task records from passive lists into decision-support objects.

Similarly, a calendar is not only a repository for appointments. It is an external memory tool for seeing upcoming work, identifying busy days, planning preparation time, and distributing workload over time [2, 3]. Therefore, task management and time planning naturally complement each other. Zaman Sayar combines these two concepts within the same application backbone.

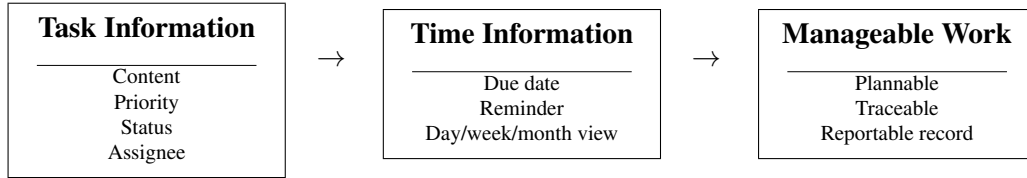


Figure 1: Conceptual relation between task management and time planning

2.2 Personal Information Management

Personal information management refers to the organization of information generated or used during daily work so that it remains findable, understandable, and reusable. Such information may be a short note, an audio record, a task description, a table row, a file attachment, a comment, or a calendar entry. The literature indicates that storing information is not sufficient; information should remain usable in later contexts [4].

When notes, audio records, task explanations, table rows, and responsibility information remain in separate systems, context becomes difficult to preserve. Zaman Sayar therefore keeps notes, audio notes, task records, calendar entries, and tables under the same working environment.

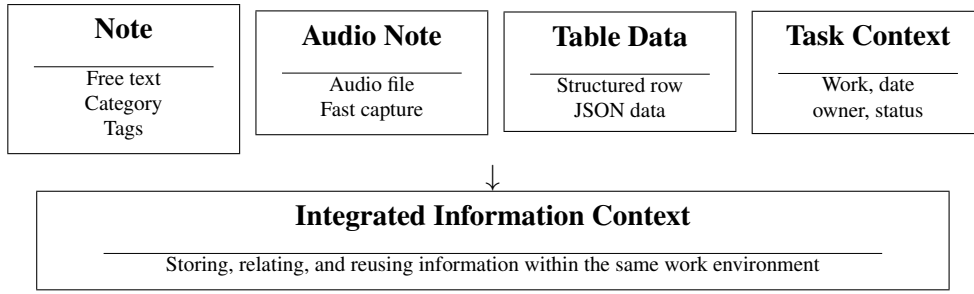


Figure 2: Preserving context in personal information management

2.3 The RACI Approach

RACI is a responsibility-assignment approach used to clarify roles in team work. It stands for *Responsible*, *Accountable*, *Consulted*, and *Informed*. The method separates the person who performs the work, the person ultimately accountable for the result, stakeholders whose input is needed, and those who should be informed [5]. In many work processes, a single assignee field is not sufficient to describe responsibility.

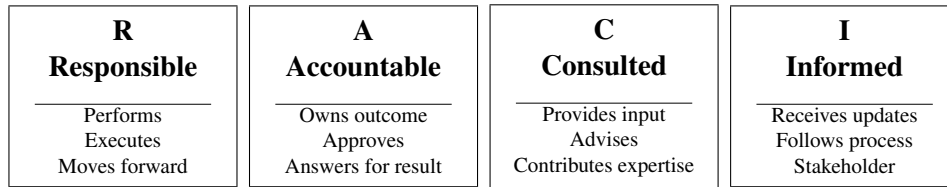


Figure 3: Meaning of RACI roles around a task

2.4 OCR-Supported Information Extraction

Optical Character Recognition (OCR) converts text in images or documents into machine-readable text. Tesseract is a long-established open-source OCR engine [6, 7], while EasyOCR provides practical multi-language OCR support in the Python ecosystem [8]. However, OCR output is often only raw text. In a task-management context, this text should be mapped into structured fields such as task content, assignee, assigner, due date, and notes.

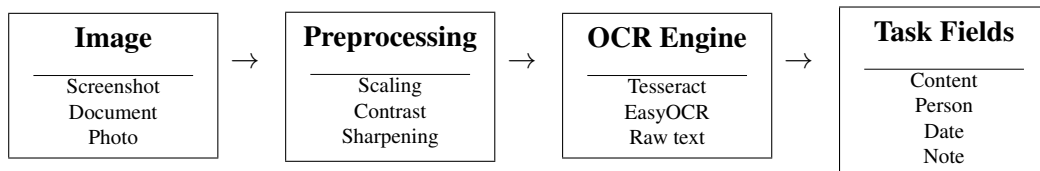


Figure 4: Conceptual flow of OCR-supported task extraction

2.5 Web-Based Software Architecture

In web applications, maintainability depends on separating presentation, business logic, and data access. ASP.NET Core MVC supports this separation through Model, View, and Controller responsibilities, while Entity Framework Core maps object-oriented models to relational databases [9, 10]. In Zaman Sayar, this structure prevents module logic from accumulating in a single file and supports future extensibility.

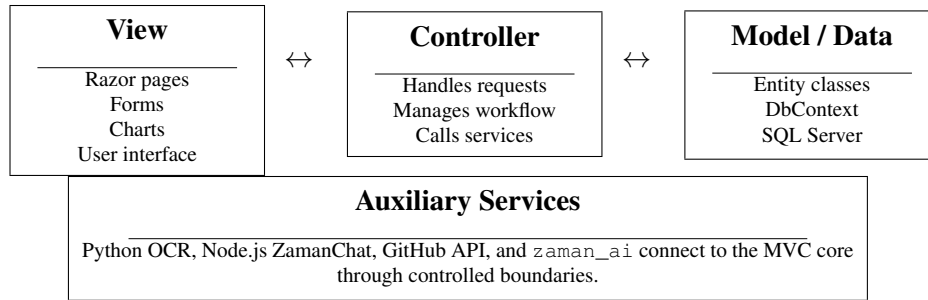


Figure 5: Conceptual MVC architecture used in Zaman Sayar

2.6 Natural-Language Interface and Read-Only Boundary

A natural-language interface allows users to ask questions without using a formal query language or a complex filter screen. ZamanChat is designed as such an auxiliary layer. Since natural-language interfaces may create data-integrity risks if they are allowed to write to the database, ZamanChat is intentionally limited to read-only behavior. It rejects intentions related to insertion, deletion, update, and schema modification.

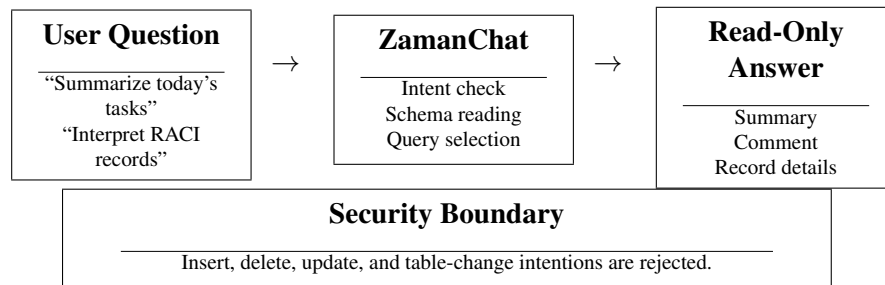


Figure 6: Natural-language interface and read-only boundary in ZamanChat

2.7 Data Model, Structured Data, and Traceability

A data model defines which entities are represented in the application and how they relate to each other. In Zaman Sayar, tasks, calendar records, notes, RACI records, dynamic tables, uploaded tables, OCR logs, GitHub settings, and suggestion feedback are modeled as separate but related entity groups. Structured fields such as dates, people, statuses, priorities, and categories make the information queryable, visualizable, and auditable.

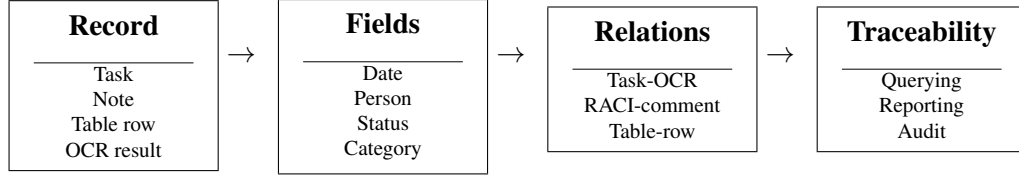


Figure 7: From structured data model to traceability

2.8 Machine Learning for Turkish Task Text Processing

Task texts are usually short, irregular, and context-dependent. In morphologically rich languages such as Turkish, the same root may appear in many surface forms, which makes automatic labeling more challenging. A common approach is to transform text into numerical features and then train supervised models. TF-IDF provides a strong baseline representation for short texts by considering both term frequency and discriminative value across the corpus [24]. Latent Semantic Analysis (LSA) can further reduce dimensionality and provide a semantic proxy representation [25]. For summarization, TextRank provides a graph-based, unsupervised, and interpretable extractive method [26].

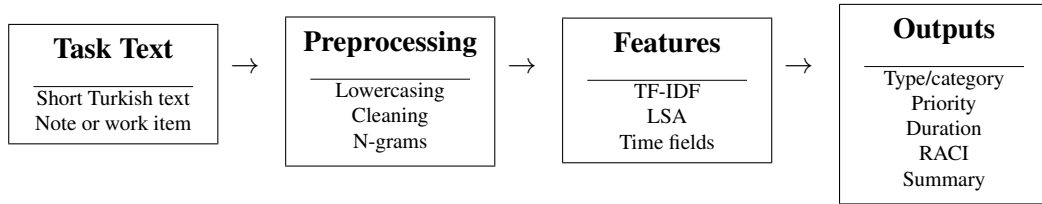


Figure 8: Feature extraction and decision-support output generation from task text

Table 2: Conceptual mapping of Zaman Sayar

Concept	General meaning	Representation in Zaman Sayar
Task management	Tracking work with status, priority, and ownership	Task and task-assignment modules
Time planning	Arranging work on a time axis	Calendar with day, week, and month views
Personal information management	Storing information together with context	Notes, audio notes, descriptions, and table records
RACI	Separating roles and responsibilities	RACI matrix and role-based views
OCR	Extracting text from images	Tesseract/EasyOCR-supported task-field generation
ORM	Mapping object-oriented models to a relational database	Entity Framework Core and ApplicationDbContext
JSON	Flexible field-based data representation	Dynamic table and uploaded-table row data
Natural-language interface	Querying data through user text	Read-only ZamanChat querying layer
Machine learning	Learning patterns from data for decision support	zaman_ai classification, duration estimation, RACI suggestion, and summarization
Traceability	Following the origin and relations of a record	OCR logs, RACI comments, GitHub settings, and suggestion feedback

3 RELATED WORK

This section discusses the academic and technical background of Zaman Sayar by relating the literature directly to the implemented modules. The project is not only a task list or only a calendar application. It combines task management, time planning, personal information management, responsibility assignment, flexible data entry, OCR-supported information extraction, natural-language data access, GitHub-based software-development traceability, and machine-learning-based decision support.

3.1 Task- and Calendar-Oriented Systems

Studies on task management and calendar systems show that users expect digital tools not only to store records, but also to help them turn work into action. Bellotti et al. report that personal task management requires context, priority, time, and next-action information rather than simple list entries [1]. Payne describes calendars as external memory tools for organizing daily work [2], while Tungare and Perez-Quinones show that personal calendar use varies by user habits and therefore requires flexible interpretation [3].

Zaman Sayar follows this literature by treating task and calendar modules as complementary views of the same work context. A task can be recorded with content, priority, owner, due date, description, and completion state, while the calendar makes the time distribution of these records visible.

3.2 Personal Information Management and Context Continuity

Personal information management research focuses on keeping information findable, understandable, and reusable. Jones defines personal information management as collecting, organizing, storing, finding, and reusing information [4]. Barreau and Nardi show that users organize information not only for archiving but also for reminding themselves [17]. Whittaker and Sidner demonstrate that communication tools often become informal task and information-management systems [18].

In Zaman Sayar, notes and audio notes are therefore not treated as isolated text fields. They are kept in the same application context as tasks, calendar records, tables, and responsibilities.

3.3 Responsibility Management and RACI

In team workflows, it is important to distinguish who performs a task, who is accountable for it, who should be consulted, and who should be informed. RACI provides a practical

responsibility-assignment structure for this purpose [5]. Traditional task tools often represent responsibility with a single assignee field, which may be insufficient for multi-stakeholder work.

The RACI module in Zaman Sayar makes responsibility distribution visible through role fields, progress values, and comments. This extends the system from individual task tracking toward team-oriented responsibility traceability.

3.4 Structured and Flexible Data Management

Sustainable software systems require consistent data models behind their interfaces. Chen’s entity-relationship model provides a foundational approach for representing real-world objects and their relationships in database design [19]. Zaman Sayar uses Entity Framework Core and SQL Server to represent tasks, users, notes, comments, OCR outputs, GitHub settings, and feedback as structured entities [10, 11].

At the same time, users may need to store data that does not fit fixed forms. Dynamic tables and spreadsheet import address this need by keeping user-defined fields and CSV/XLSX rows as JSON-based flexible data while preserving the relational backbone [12].

3.5 OCR-Supported Information Extraction

OCR systems convert image-based text into machine-readable form. Tesseract is a well-known open-source OCR engine [6, 7], and EasyOCR provides a practical Python-based solution for multilingual text recognition [8]. However, task-management use cases require more than raw OCR text. The extracted text should be mapped to task content, assignee, assigner, due date, and notes.

Zaman Sayar implements this distinction explicitly. The Python OCR workflow performs image preprocessing, text extraction, field matching, confidence scoring, and task-draft generation. The raw text, engine information, confidence score, and generated task link are stored for traceability.

3.6 Natural-Language Interfaces and Safe Data Access

Natural-language interfaces to databases aim to reduce the technical distance between users and data, but they introduce challenges related to ambiguity, schema matching, and user intent [20]. Interactive natural-language interfaces also require a meaningful mapping between user questions and relational database structures [21].

ZamanChat addresses this issue through a strict read-only boundary. It reads database schema information, identifies the relevant module and operation type, and returns sum-

maries, interpretations, or record details without performing insert, update, delete, or schema-changing operations.

3.7 Web Architecture, Layered Design, and Maintainability

Layered architecture is a common strategy for managing complexity in enterprise applications [22]. ASP.NET Core MVC supports this by separating model, view, and controller responsibilities [9]. In Zaman Sayar, Razor views produce the interface, controllers manage request flow, Entity Framework Core handles data access, SQL Server stores persistent data, Python handles OCR and machine-learning tasks, and Node.js provides the ZamanChat service [15, 16].

From a software-quality perspective, ISO/IEC 25010 provides a framework for evaluating functional suitability, maintainability, usability, reliability, and security [23]. Zaman Sayar addresses these quality aspects through modularity, traceability, and extensibility, while production-level authorization, broader automated testing, monitoring, and usability hardening remain future work.

3.8 Machine Learning for Task Text Classification and Summarization

For short work or task texts, TF-IDF vectorization with linear models is known to provide a strong baseline [24, 27]. Latent Semantic Analysis can capture semantic similarity in a lower-dimensional representation [25]. Tree-based boosting methods such as XGBoost and LightGBM are widely used for structured feature data [28, 29]. For extractive summarization, TextRank ranks sentences in a graph and provides an interpretable unsupervised method [26]. Clustering metrics such as Silhouette, Calinski-Harabasz, Davies-Bouldin, ARI, and NMI provide information about internal structure and alignment with external labels [30–33].

Zaman Sayar uses these methods not as an isolated machine-learning study, but as a decision-support layer connected to a working CRM/time-management product. It brings task classification, duration estimation, RACI suggestion, summarization, and feedback capture into a single measurable application workflow.

3.9 Design Implications from the Literature

Table 3: Design implications derived from the literature

Literature area	Requirement	Zaman Sayar implementation
Task management	Tasks should be tracked with priority and state	Task and task-assignment modules
Calendar use	Work should be visible on a time axis	Day, week, and month calendar views
Personal information management	Notes and supporting data should remain findable	Notes, audio notes, categories, and tags
Responsibility assignment	Role relations should not be reduced to a single assignee	RACI matrix, role fields, and comments
Data modeling	Records should be related, consistent, and traceable	EF Core, SQL Server, and relational entities
Flexible data entry	Users should be able to store custom structured data	Dynamic tables and spreadsheet import
OCR technologies	Image text should be mapped to application fields	OCR service with field extraction and logging
Natural-language interfaces	Data access should be easy but safe	Read-only ZamanChat layer
Machine learning	Short task text can support classification and prediction	<code>zaman_ai</code> intelligent suggestion workflow

3.10 Positioning of Zaman Sayar

The main distinction of Zaman Sayar is not specialization in a single module, but the integration of multiple workflows within the same data context. Existing tools may be strong as calendars, task lists, note applications, spreadsheets, OCR utilities, or chat interfaces. However, when users want to manage these workflows under the same project context, fragmentation and repeated data entry may emerge.

Table 4: Comparison of common tool approaches and Zaman Sayar

Approach	Strength	Limitation alone	Added value of Zaman Sayar
Calendar tools	Time visibility and appointment tracking	Limited task details and responsibility context	Calendar connected to tasks, notes, RACI, and statistics
Task lists	Tracking work items with state and priority	Time, OCR, table, and responsibility data may remain separate	Subtasks, task assignment, OCR, and calendar connection
Note tools	Free text and fast information capture	Notes may remain disconnected from workflow	Notes stored with the same work context
Spreadsheets	Flexible data organization	Not directly tied to task/time management	Dynamic and uploaded tables inside the application
RACI templates	Role-based responsibility separation	Often static documents or separate files	Live RACI records with progress and comments
OCR tools	Raw text extraction from images	Extracted text must still be structured	Raw text converted into task fields and logs
Chat interfaces	Natural-language access to information	Uncontrolled write authority may risk data integrity	Read-only, schema-aware, bounded interaction model
Machine-learning tools	Classification and prediction support	May remain detached from product workflow	Intelligent suggestions linked to interface, feedback, and measurement

4 MATERIALS AND METHOD

4.1 Development Approach

Zaman Sayar was developed through an incremental approach in which requirements were divided into modules, each module was implemented with its own data model and user-interface flow, and all modules were later integrated through a common database and application shell. The initial development focused on core task and calendar flows, followed by notes, RACI, dynamic tables, spreadsheet import, statistics, OCR, ZamanChat, GitHub integration, and the `zaman_ai` machine-learning subsystem.

Methodologically, the study follows three axes: transforming user needs into modules, building a layered software architecture to support these modules, and evaluating the outputs in terms of interface, data model, algorithms, measurements, and limitations.

4.2 System Requirements

Functional requirements were derived from the need to create work records, relate them to time, support them with notes, define responsibilities, work with structured data, and later interpret stored records. Non-functional requirements focus on maintainability, modularity, traceability, data integrity, and extensibility.

Table 5: System requirements of Zaman Sayar

Requirement class	Requirement	Component
Planning	Tracking tasks in date and time context	Calendar, daily plan panel
Task tracking	Status, priority, assignee, and sub-tasks	Task and task-assignment modules
Information capture	Written and audio notes	Note module, MediaRecorder support
Responsibility management	Separating R, A, C, and I roles	RACI matrix and role filters
Flexible data	User-defined tables and file import	Dynamic tables, spreadsheet import
Automation	Generating task fields from images	Python OCR service
Information access	Natural-language record reading	Read-only ZamanChat service
Visualization	Tracking completion and distribution	Statistics screens, Chart.js
Software traceability	Linking development artifacts to planning	GitHub integration
Decision support	Generating intelligent suggestions from task text	<code>zaman_ai</code> subsystem

4.3 Software Architecture

The application runs on a layered web architecture. The main application is implemented with ASP.NET Core MVC; Razor views generate the user interface; Entity Framework Core performs data-access operations on SQL Server; and auxiliary components provide OCR, natural-language querying, GitHub integration, and machine-learning inference [9–11]. Figure 9 summarizes the architecture.

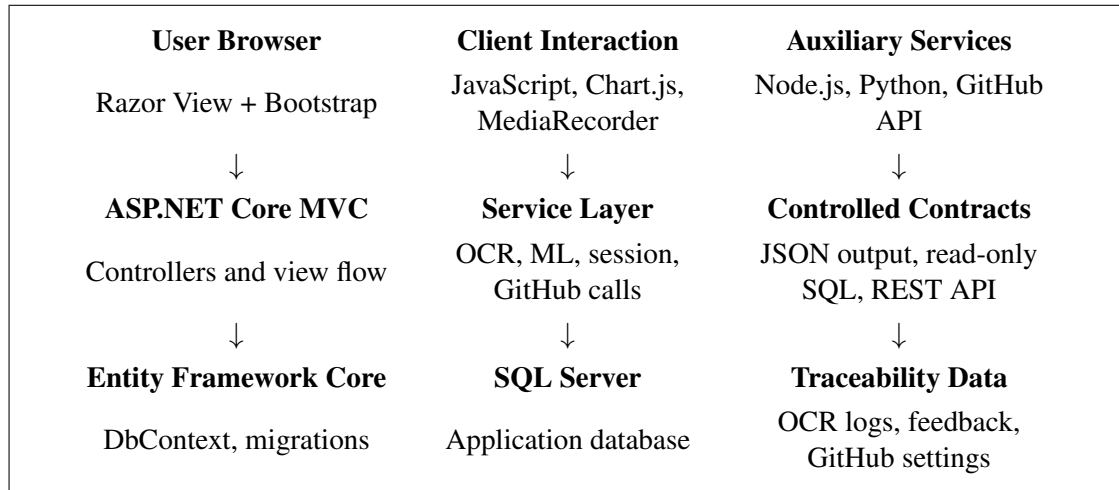


Figure 9: Software architecture of Zaman Sayar

Table 6: Main technologies used in the architecture

Layer	Technology	Purpose
Web application	ASP.NET Core MVC, Razor	Controller-based workflow and server-side page generation
Data access	Entity Framework Core 9, SQL Server	Storing entity classes in a relational database
Client side	Bootstrap, JavaScript, Chart.js	Interface interactions and chart visualization [13, 14]
File handling	EPPlus, CSV processing	Importing and exporting spreadsheet data [12]
OCR	Python, Pillow, Tesseract, EasyOCR	Image preprocessing and text extraction
Chat service	Node.js, Express, Socket.IO, MSSQL driver	Read-only natural-language data querying
Machine learning	Python, scikit-learn, pandas, NumPy, joblib	Classification, duration estimation, RACI suggestion, summarization
GitHub integration	GitHub REST API, HttpClientFactory	Repository, issue, pull request, activity, and calendar-synchronization data

4.4 Project Structure

The source code is organized into four .NET projects and two auxiliary service areas. CRM_9 contains the web application, controllers, Razor views, services, and static files. CRM.Models stores entity classes and view models. CRM.DataAccess contains the database context and migration files. CRM.Utility is reserved for shared constants and helpers. Auxiliary components include Python/ocr_to_task.py, Python/zaman_ai, and NodeBot/server.js.

Table 7: Controller responsibility map

Controller	Main responsibility
HomeController	Main page and daily plan panel.
TaskController	Personal tasks, subtasks, completion state, and soft-delete flow.
TaskAssignmentController	Detailed task assignment and OCR-based task creation.
CalendarController	Calendar tasks in day, week, and month contexts.
NoteController	Text notes, categories, tags, and audio notes.
DynamicBoardController	RACI matrix, comments, and role-based summaries.
DynamicTableController	User-defined dynamic tables and JSON row data.
DynamicTablesController	Excel/CSV upload, header update, row editing, and export.
StatisticsController	Completion, distribution, and busy-day statistics.
SuggestionController	Intelligent suggestions from zaman_ai and feedback capture.
GitHubController	GitHub settings, repository/issue/PR views, dashboard, and calendar sync.

4.5 Data Model

The data model consists of Identity-based user infrastructure and module-specific application tables. Task-assignment records may be related to OCR logs; task items can form parent-child subtask relations; RACI board items can be extended with comments; dynamic and uploaded tables store row data as JSON; GitHub settings are scoped to an owner; and intelligent-suggestion feedback is stored for later inspection and acceptance-rate measurement.

Entity	Relation and purpose
TaskAssignments	Stores detailed task-assignment records and can relate to OcrTaskLogs.
OcrTaskLogs	Stores OCR engine, raw text, confidence, and generated task trace.
TaskItems	Contains personal tasks and parent-child subtask relations.
CalendarTasks	Stores time planning, recurrence, reminder, and color-tag data.
Notes	Stores note text, category, tags, and audio-note path.
DynamicBoardItems	Stores RACI tasks, role distribution, and progress.
DynamicBoardComments	Stores comments linked to RACI records.
DynamicTableDefinitions / DynamicTableData	Store user-defined table fields and JSON row data.
UploadedTables / UploadedRows	Store imported spreadsheet roots and JSON rows.
GitHubSettings	Stores user/team GitHub access settings.
SuggestionFeedback	Stores ML outputs, decisions, and human-in-the-loop feedback.

Figure 10: Summary of the Zaman Sayar data model

4.6 Module Design

Table 8: Module-based design summary

Module	Main data structure	Engineering value
Calendar	CalendarTask	Tracks tasks by date, recurrence, reminder, and color.
Tasks	TaskItem	Provides subtasks, completion, reminder, and soft-delete support.
Task Assignment	TaskAssignment	Stores assigner, assignee, priority, tracking, and completion data.
Notes	Note	Stores text, category, tags, and audio notes in the same record.
RACI	DynamicBoardItem	Makes R, A, C, I roles and progress visible.
Dynamic Table	DynamicTableDefinition, DynamicTableData	Provides flexible tables with user-defined fields.
Spreadsheet Import	UploadedTable, UploadedRow	Converts CSV/XLSX files into JSON row data.
Statistics	StatisticsViewModel	Visualizes task distribution and workload.
OCR	OcrTaskLog, OcrTaskExtractionResult	Produces task fields from images and preserves raw output.
ZamanChat	Live SQL schema reading	Summarizes records in a read-only natural-language layer.
GitHub	GitHubSetting, CalendarTask	Links repository, issue, PR, and milestone due-date data to planning.
Intelligent Suggestion	SuggestionFeedback, TaskSuggestionResult	Generates ML-supported type, category, priority, duration, RACI, and summary outputs.

4.7 OCR Workflow

The OCR component is designed in two stages. First, the image is prepared and text is extracted with Tesseract and EasyOCR. Second, the raw text is converted into task fields using Turkish and English label synonyms. The system does not simply select the longest OCR output; it aims to select the output most useful for task-field extraction.

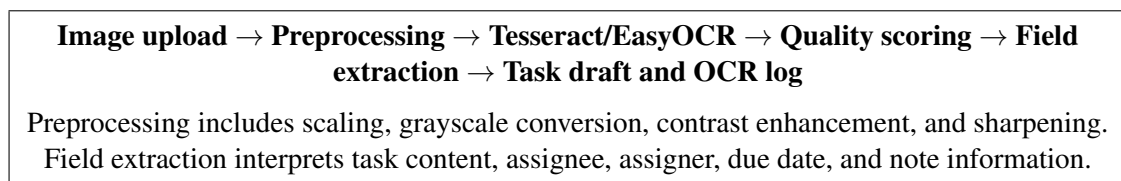


Figure 11: OCR-supported task extraction workflow

The engine-selection concept can be represented by Equation 1, where F_m is the number of extracted fields, E_m is label-matching success, Q_m is raw-text readability, and C_m is engine confidence.

$$S_m = 0.35F_m + 0.25E_m + 0.20Q_m + 0.20C_m \quad (1)$$

4.8 ZamanChat Workflow and Security Boundary

ZamanChat is an auxiliary component designed to make data access easier through natural language. It reads table and column information from `INFORMATION_SCHEMA`. It searches the user message for tab names, operation type, and record identifiers. The operation type is limited to summarizing, interpreting, or writing record details. If a write-oriented intention is detected, the service refuses the operation.

Table 9: ZamanChat decision flow

Step	Operation
Input cleaning	User text is normalized and empty input is rejected.
Write-intent check	Insert, delete, update, drop, or alter intentions are detected.
Action detection	Summary, interpretation, or detail-writing actions are identified.
Profile detection	Calendar, task assignment, note, RACI, and table tabs are matched.
Record selection	Date is expected for calendar queries; ID is expected for other profiles.
Query execution	Parameterized and limited SELECT queries are executed.
Response generation	Results are returned as summaries, comments, or details.

4.9 GitHub Integration

A *GitHub integration* was added to connect software-development traceability with task management in the same interface (Figure 12). The integration uses the GitHub REST API (`api.github.com`, API version 2022-11-28) and authenticates with a user/team-scoped Personal Access Token (PAT). All calls are associated with the current multi-tenant `OwnerId` key and are performed through an `HttpClient` managed by `IHttpClientFactory`, using `Bearer` authorization and the required `GitHub User-Agent` header.

The integration is designed in three stages:

1. **Profile and repositories:** The connected account profile and repository list are retrieved and displayed in card form, with recently updated repositories prioritized.
2. **Issues and pull requests:** The GitHub Search API is used to filter issues and pull requests assigned to, authored by, or mentioning the user. Open issues with mile-

stone due dates can be synchronized into the application calendar as non-duplicated `CalendarTask` records.

3. **Dashboard, activity, and releases:** Repository count, stars, forks, open issues, language distribution, most-starred repositories, recently updated repositories, daily activity for the last 14 days, and release records are summarized.

All API calls are wrapped defensively. In the case of an invalid token or network failure, the user receives an informative message and the interface is protected from crashing. Because the `EF Core DbContext` is not thread-safe, dashboard calls that depend on database context are intentionally executed sequentially. This is a measured trade-off between correctness and response time.

Known limitation. In the current prototype, the personal access token is stored in the database without encryption. In a production deployment, it should be protected with ASP.NET Core Data Protection or a secret-management service such as Azure Key Vault.

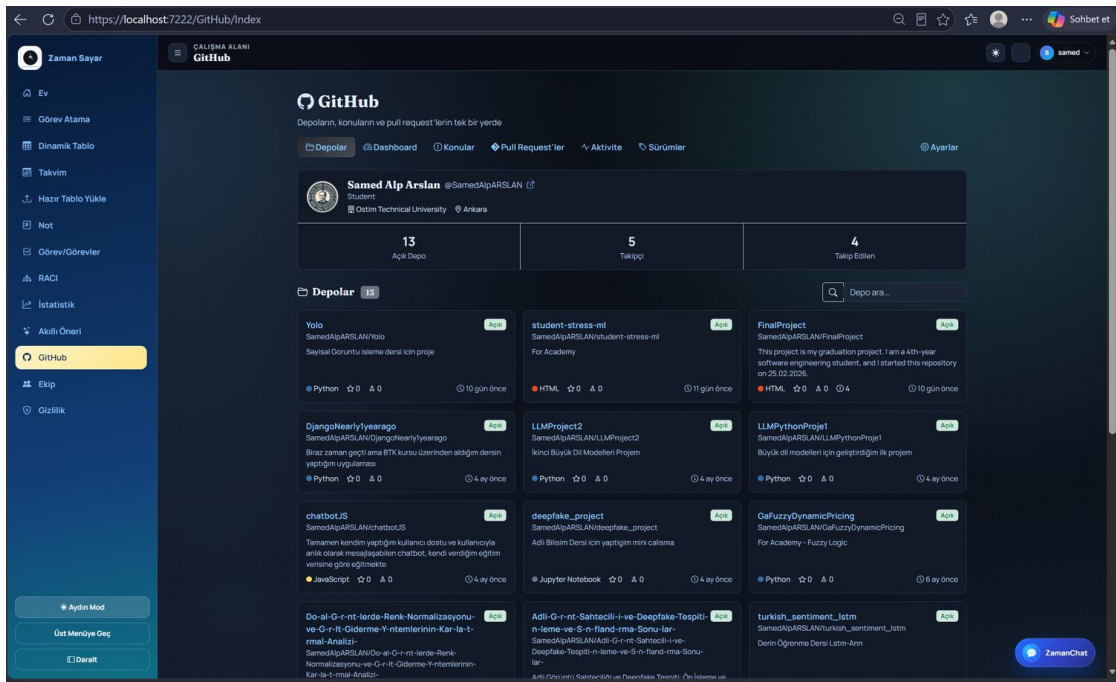


Figure 12: In-application GitHub integration: connected account profile, repository cards, language labels, and navigation tabs for repositories, dashboard, issues, pull requests, activity, and releases.

4.10 Machine-Learning Subsystem (`zaman_ai`)

The `zaman_ai` subsystem was added as an independent Python package without changing the core application architecture. It includes preprocessing, dataset generation, feature extraction, classification (A1), duration/lateness estimation (A2), summarization (A3), RACI suggestion (A4), experiment logging, and clustering (EK-3). The C# side uses the same

process-based contract as the OCR component: `predict.py` outputs a single JSON object on the last line, and the ASP.NET Core service layer converts it into an application model.

Table 10: zaman_ai modules and research goals

Module	Goal	Task
text_processing.py	Common	Turkish text cleaning, normalization, and light morphological simplification.
dataset.py	A5	Synthetic and anonymized Turkish task dataset generation.
features.py	A1–A4	TF-IDF, LSA, temporal, and workload feature extraction.
classifier.py	A1	Task type, category, and priority classification.
deadline.py	A2	Duration estimation, reminder timing, and lateness simulation.
summarizer.py	A3	TextRank-based extractive summarization and readability evaluation.
raci.py	A4	RACI role suggestion and acceptance-rate measurement.
clustering.py	EK-3	Task-text clustering and external-alignment metrics.
run_experiments.py	A5	Running all experiments and producing CSV/JSON/-Markdown reports.

4.11 Dataset and Ethics

The models were developed on a synthetic and anonymized Turkish task dataset in order to avoid the use of real personal data. The dataset contains 1200 records. Instead of personal names, role labels such as `gelistirici`, `test_uzmani`, `tasarimci`, `analist`, `destek_uzmani`, `yonetici`, and `ekip_lideri` are used. Each record includes task text, type, category, priority, creation/due/completion dates, estimated and actual duration, assigned role, workload, RACI role, and lateness.

The dataset was calibrated to represent a moderately learnable regime rather than an artificially perfect separation. This allows both strengths and limitations of the models to be observed. Measurement reports, an open ER schema, and anonymized sample data were preserved under the A5 target.

4.12 Feature Extraction and Models

Text features are extracted with TF-IDF using 1–2 grams, `min_df=2`, `max_df=0.95`, `sublinear_tf`, and a maximum of 5000 features. LSA based on TruncatedSVD is used as a semantic proxy representation. Temporal features include estimated duration, workload, priority order, day of week, weekend indicator, and encoded task type.

Three classification approaches are compared: rule baseline, Logistic Regression, and histogram-based gradient boosting. Duration estimation is treated as a regression problem, while RACI recommendation is treated as a multi-class classification problem.

4.13 Evaluation Protocol

All models are evaluated with 5-fold stratified cross-validation where applicable. Classification tasks report accuracy, macro-F1, weighted F1, and ROC-AUC; regression reports MAE, MAPE, and R^2 ; clustering reports Silhouette, Calinski-Harabasz, Davies-Bouldin, ARI, and NMI. Paired t-test, Wilcoxon signed-rank test, Cohen's d , and 95% confidence intervals are stored in the measurement reports.

4.14 Evaluation Indicators

Task completion rate is defined in Equation 2.

$$T = \frac{N_{completed}}{N_{total}} \times 100 \quad (2)$$

Daily workload can be modeled as shown in Equation 3, where $w(p_i)$ is the priority weight and c_i is the completion state.

$$Y(d) = \sum_{i=1}^n I(date_i = d) \cdot w(p_i) \cdot (1 - c_i) \quad (3)$$

RACI role coverage is expressed in Equation 4.

$$C_{RACI} = \frac{r_R + r_A + r_C + r_I}{4} \quad (4)$$

For machine-learning evaluation, macro-F1 is used as the mean of class-level F1 values:

$$F1_{macro} = \frac{1}{K} \sum_{k=1}^K \frac{2P_k R_k}{P_k + R_k} \quad (5)$$

Mean absolute percentage error and lateness reduction are defined in Equations 6 and 7.

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (6)$$

$$\Delta = 100 \times \frac{L_{baseline} - L_{model}}{L_{baseline}} \quad (7)$$

The A4 acceptance rate is defined as the percentage agreement between out-of-fold RACI predictions and approved labels.

5 FINDINGS AND DISCUSSION

5.1 Application Outputs

The study produced a multi-module web-based time and task management application named Zaman Sayar. The main screen welcomes the user with a single working panel and provides access to task assignment, dynamic tables, calendar, spreadsheet import, notes, tasks, RACI, statistics, intelligent suggestions, GitHub integration, and team functions through the navigation menu. The main page also includes the daily plan panel and the ZamanChat component.

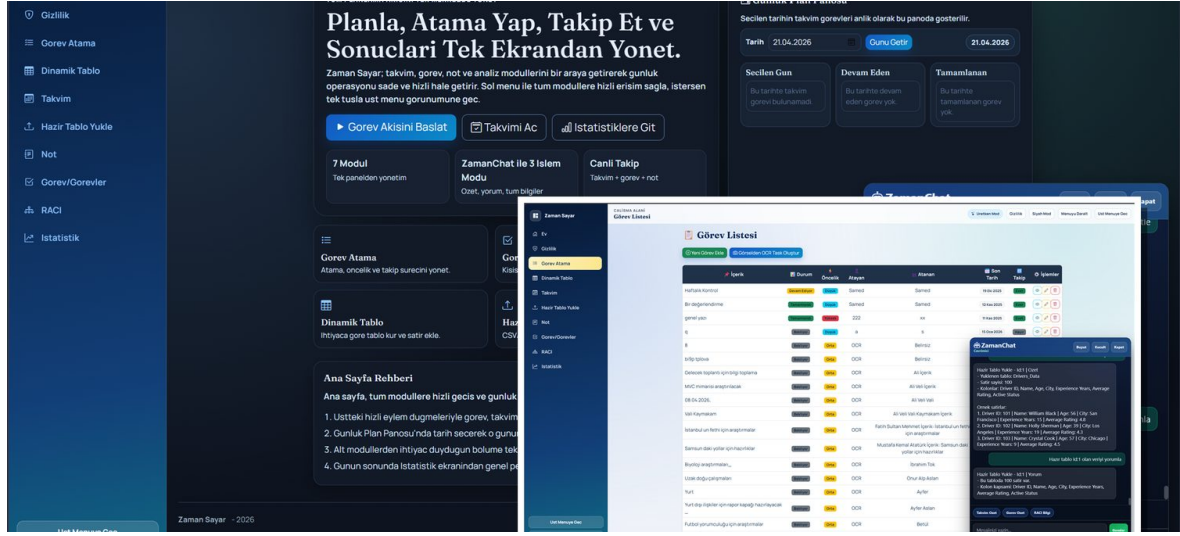


Figure 13: Zaman Sayar main panel, task list, and ZamanChat view

The source code was compiled locally with the .NET 9 SDK before the reporting phase and provided a working technical basis. Production-level maturity still requires broader validation, monitoring, security hardening, and user-experience refinement.

5.2 Functional Verification

Functional evaluation was carried out by checking whether each module performs its expected core function and how it contributes to the overall application.

Table 11: Functional verification summary

Module	Verified function	Evaluation
Main panel	Fast access to modules and daily plan panel	Provides an integrated entry point.
Task Assignment	Assigner, assignee, priority, date, and tracking fields	Supports formal task tracking.
Tasks	Subtasks, completion, and reminders	Supports personal work breakdown and monitoring.
Calendar	Monthly, weekly, and daily time views	Makes time context visible.
Notes	Text, category, tags, and audio note	Keeps contextual information within the same system.
RACI	R, A, C, I roles and comments	Makes responsibility sharing visible.
Dynamic table	User-defined fields and JSON row data	Provides flexible structured data storage.
Spreadsheet import	Excel/CSV upload and export	Allows external data to be brought into the system.
OCR	Extracting task fields from images	Reduces manual data entry.
ZamanChat	Read-only natural-language querying	Provides faster data access while preventing write operations.
GitHub	Repository, issue, PR, activity, and calendar sync	Links software-development artifacts to time planning.
Intelligent Suggestion	ML-based suggestion from task text	Provides type/category/priority, duration, RACI, and summary outputs.

5.3 A1: Task Type, Category, and Priority Classification

Under A1, task type, category, and priority were evaluated as separate classification targets on 1200 synthetic and anonymized records with 5-fold cross-validation.

Table 12: A1 – Classification performance (1200 records, 5-fold cross-validation)

Target	Model	Accuracy	Macro-F1	Weighted F1	ROC-AUC
Type	Rule baseline	0.8342	0.8408	0.8388	0.500
Type	LogReg	0.9117	0.9116	0.9118	0.9926
Type	HistGB	0.9042	0.9046	0.9043	0.9869
Category	Rule baseline	0.9058	0.9097	0.9068	0.500
Category	LogReg	0.9183	0.9163	0.9183	0.9933
Category	HistGB	0.9183	0.9162	0.9182	0.9879
Priority	Rule baseline	0.8750	0.8766	0.8727	0.500
Priority	LogReg	0.9308	0.9273	0.9313	0.9941
Priority	HistGB	0.9142	0.9129	0.9142	0.9884

Logistic Regression was the best model for all three targets. Macro-F1 scores between 0.91 and 0.93 clearly exceed the 0.85 target. For task type classification, the mean F1 difference between Logistic Regression and the rule baseline was 0.071, with paired t-test $p = 0.00248$, Cohen’s $d = 3.03$, and 95% confidence interval [0.898; 0.925].

5.4 A2: Duration Estimation and Lateness Reduction

A2 aims to estimate task duration and reduce lateness by using the model outputs in a budget-matched simulation.

Table 13: A2 – Duration estimation performance (5-fold cross-validation)

Model	MAE (hours)	MAPE (%)	R^2
Baseline estimate	2.4025	30.51	0.5949
Linear	1.3703	22.43	0.8624
HistGB	1.1973	17.55	0.9045

Table 14: A2 – Budget-matched lateness reduction simulation

Indicator	Value
Average lateness – baseline (hours)	1.0777
Average lateness – model (hours)	0.6000
Lateness reduction (%)	44.32
Target	$\geq 20\%$
Status	PASSED

The best duration model (HistGB) achieved 1.20 hours MAE and 17.6% MAPE. In the budget-matched simulation, the average lateness decreased from 1.08 hours to 0.60 hours, corresponding to a 44.3% reduction.

5.5 A3: Note Summarization

The summarization component uses a TextRank-based extractive method. Table 15 presents the automatic proxy readability results, while Table 16 reports the primary AI-assisted/simulated Likert panel.

Table 15: A3 – TextRank summary quality (automatic proxy readability, 1–5)

Note	Method	Coverage	Conciseness	Non-repetition	Proxy Score
1	textrank	0.529	1.00	1.00	4.059
2	textrank	0.500	1.00	0.933	3.947
3	textrank	0.400	1.00	1.00	3.800
4	textrank	0.417	1.00	0.800	3.673
5	textrank	0.367	1.00	0.909	3.661
Mean: 3.828 / 5 (95% CI [3.613; 4.043])					

Table 16: A3 – Readability Likert panel (N=20, AI-assisted/simulated; primary measure)

Indicator	Value (95% CI)	Threshold	Status
Clarity/readability – primary	4.50/5 [4.38; 4.62]	$\geq 4/5$	PASSED
Composite score (5 dimensions)	4.08/5 [3.98; 4.18]	$\geq 4/5$	PASSED
Cronbach α consistency	0.765	≥ 0.70	Acceptable
Dimensions: clarity 4.50; faithfulness 4.79; conciseness 3.99; coherence 3.88; coverage 3.25			

The primary readability score reached 4.50/5 with a 95% confidence interval of [4.38; 4.62]. However, this panel is not a substitute for a real 20-participant human-subject study. The produced summaries were first scored against source notes and then 20 simulated evaluators were generated around those anchors with fixed random seed 42. Therefore, the result is reported transparently as an AI-assisted/simulated panel. The weakest dimension is coverage (3.25), indicating that some secondary requests or decisions may be omitted by extractive summarization.

5.6 A4: RACI Responsibility Suggestion

Table 17: A4 – RACI suggestion performance and acceptance rate (5-fold cross-validation)

Model	Accuracy	Macro-F1	Acceptance Rate (%)
Rule baseline	0.6200	0.3974	62.00
LogReg	0.7125	0.7168	71.25
HistGB	0.8908	0.8590	89.08

The best RACI model reached an 89.08% acceptance rate, defined as the agreement between out-of-fold prediction and approved label. In live use, user decisions are stored in the `SuggestionFeedback` table, enabling continuous measurement.

5.7 EK-3: Task Text Clustering

Table 18: EK-3 – Task text clustering metrics

Algorithm	Silhouette	Calinski-Harabasz	Davies-Bouldin	Clusters
K-Means (k=8)	0.0623	26.43	3.5073	8
DBSCAN (eps≈0.75)	–	–	–	1
External alignment: ARI = 0.0053; NMI = 0.0366				

The clustering experiment produced weak separation. The Silhouette coefficient was 0.062, and external alignment was low (ARI 0.005; NMI 0.037). This negative finding is reported as-is and supports the conclusion that supervised classification is more suitable than unsupervised clustering for the short Turkish task texts used in this study.

5.8 A5: Prototype, Measurement Report, ER Schema, and Anonymous Data

Under A5, the working prototype is represented by the *Intelligent Suggestion* module in the web interface. Machine-readable measurement reports, the ER schema including `SuggestionFeedback`, and anonymized sample data were preserved. Feedback decisions (accepted/deferred/rejected) are stored permanently and can be inspected through the interface.

5.9 Intelligent Task Suggestion Interface (Prototype)

The user-facing layer of the machine-learning subsystem (`zaman_ai`) is the *Intelligent Suggestion* page in the application navigation menu (Figure 14). This page materializes the working prototype promised under A5. The user enters free-form Turkish task or note text, and the system runs the trained models through a single inference call to generate an integrated recommendation. Thus, the A1–A4 goals are represented not only as offline metrics, but also as an interactive and auditable decision-support workflow.

The interface has two columns. The form on the left collects the task text together with contextual features such as assigned role, workload (0–10), and day of week (0=Monday). When the user requests a suggestion, the ASP.NET Core controller forwards the request to the `predict.py` script. The script follows the same process-based contract as the OCR component and returns a single JSON object on its final output line. The right column parses and displays:

- **Type, category, and priority (A1)** with labels and confidence scores;
- **Estimated duration and reminder time (A2);**

- **RACI responsibility role (A4);**
- **Extractive summary (A3) and model version.**

The bottom section provides a human-in-the-loop feedback mechanism. Each suggestion can be marked as *accepted*, *deferred*, or *rejected*. These decisions are stored in the `SuggestionFeedback` table and summarized as a live acceptance rate. Recent feedback records can be inspected with their full input text, A1–A4 outputs, confidence values, A3 summary, and decision. Due to multi-tenant `OwnerId` isolation, only the team that created a record can remove it.

For security, free-form text is passed through `ProcessStartInfo.ArgumentList` to avoid shell injection, and database access is filtered by `OwnerId`.

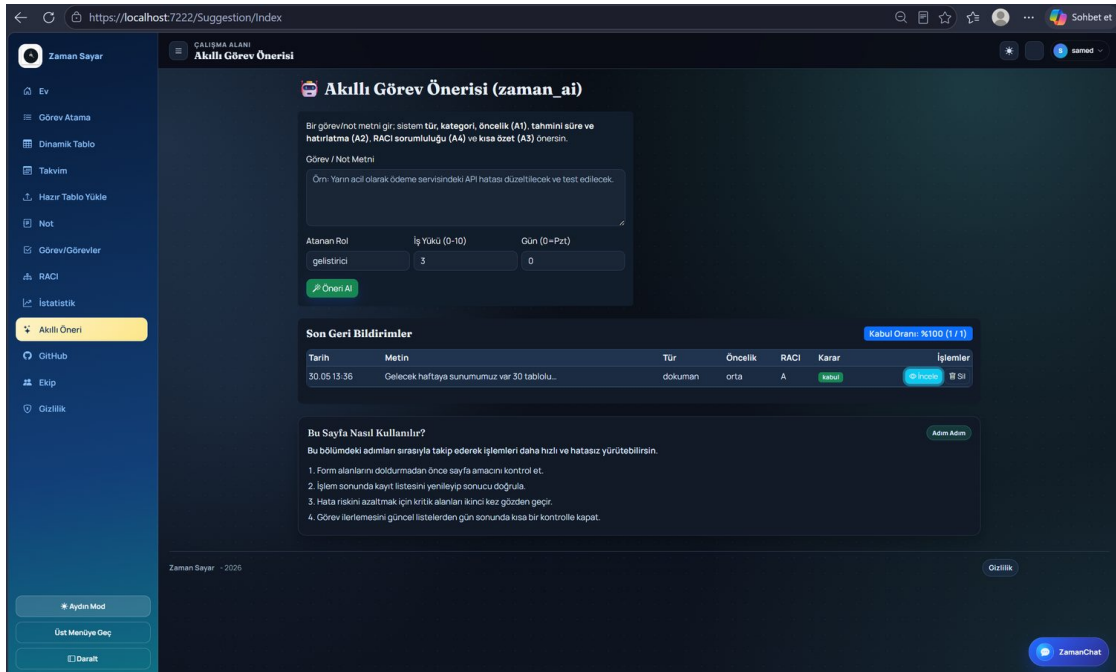


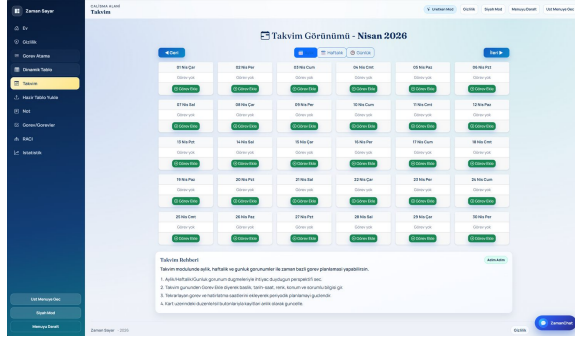
Figure 14: Intelligent Task Suggestion interface: generating task type/category/priority (A1), estimated duration/reminder (A2), RACI responsibility (A4), and extractive summary (A3), together with human-in-the-loop feedback.

Table 19: Summary of TUBITAK 2209-A target achievement

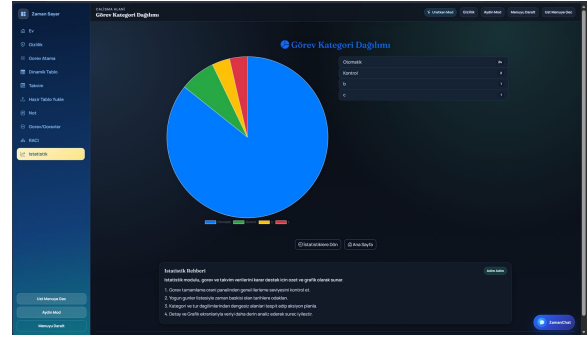
Target	Committed measure	Result	Status
A1	Macro-F1 ≥ 0.85	0.91 / 0.92 / 0.93	PASSED
A2	Lateness reduction $\geq 20\%$	44.32% (MAE 1.20 h)	PASSED
A3	Readability $\geq 4/5$	4.50/5 (N=20 AI-assisted panel)	PASSED
A4	Acceptance rate $\geq 60\%$	89.08%	PASSED
A5	Prototype+measurement+ER+data	All delivered	PASSED
EK-3	Clustering metric report	Silhouette 0.062 (weak)	REPORTED

5.10 Interface Results

The primary interface goal is to allow users to continue work without getting lost between modules. Therefore, the navigation structure is kept consistent, modules are separated through cards and forms, and daily plans and charts help users see the general state quickly.

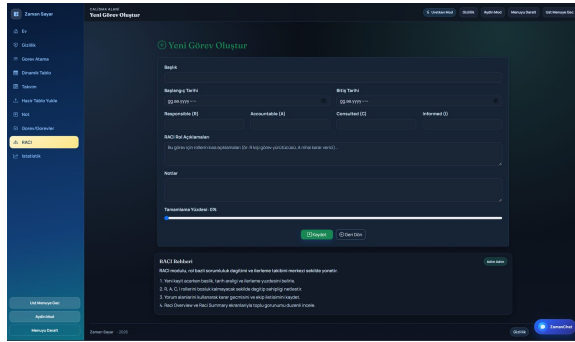


(a) Calendar module

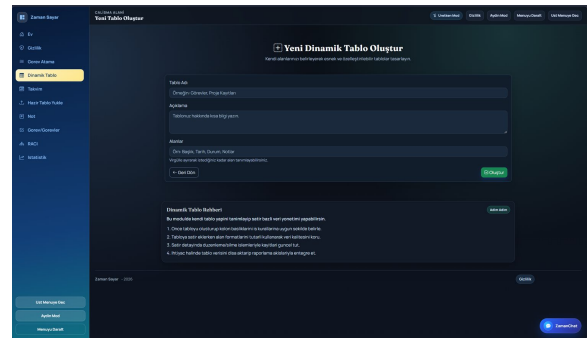


(b) Statistics module

Figure 15: Calendar and statistics modules

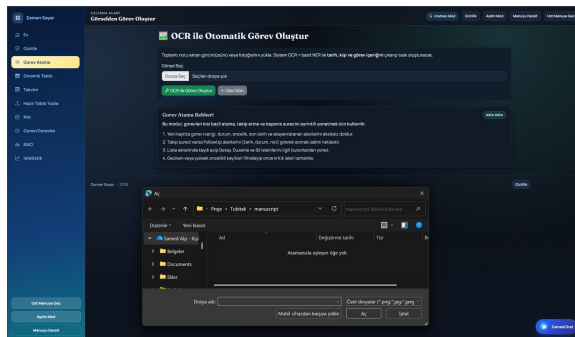


(a) RACI task creation

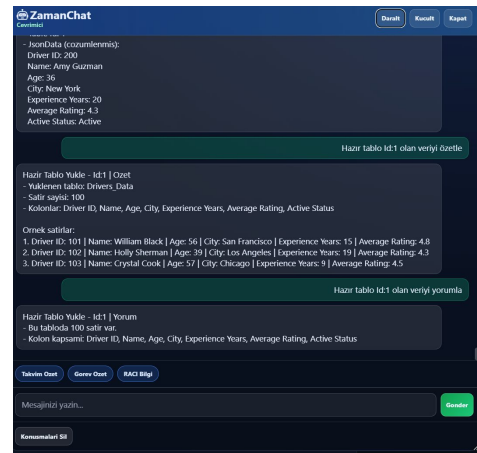


(b) Dynamic table creation

Figure 16: RACI and dynamic table interfaces



(a) OCR-based task creation



(b)

ZamanChat query

Figure 17: OCR and ZamanChat support screens

5.11 Engineering Strengths

The strongest aspect of the study is the integration of different modules around the same problem domain. Calendar, task, note, RACI, table, OCR, GitHub, ZamanChat, and intelligent suggestion components provide continuity of context. The second strength is the controlled integration of auxiliary services. OCR is executed as a Python workflow, ZamanChat is isolated as a Node.js service, GitHub communication is wrapped through an application service, and `zaman_ai` is called through a process-based JSON contract. The third strength is traceability: OCR logs, RACI comments, JSON table rows, GitHub settings, and suggestion feedback preserve the origin and interpretation of records.

5.12 Limitations

The study has several limitations. First, although Identity infrastructure exists, production-level role-based authorization is not yet complete. Second, although 55 unit tests were written for the machine-learning subsystem, end-to-end integration tests and broader test coverage remain future work. Third, null-safety, form validation, and unexpected empty-value scenarios require further production-hardening. Fourth, OCR and ZamanChat are functional but could be improved with richer error handling, natural-language understanding, and user feedback.

Fifth, the machine-learning results are based on synthetic data and should be revalidated on real field data. Sixth, the A3 readability criterion was met through an AI-assisted/simulated panel rather than a real human-subject study. A real Likert-based readability study is planned as future work. Seventh, task-text clustering showed weak separation, which is a transparent negative finding. Eighth, the GitHub personal access token is stored unencrypted in the current prototype and should be protected by Data Protection or a secret-management service in production.

5.13 Discussion

The academic strength of Zaman Sayar is that it does not remain at the level of “an application was built”. The system is explainable through its architecture, data model, service boundaries, and measurement results. MVC, EF Core, Python OCR, Node.js ZamanChat, GitHub integration, and the `zaman_ai` subsystem demonstrate the integration of multiple technologies around a coherent engineering objective.

At the same time, the artificial-intelligence claims of the system are intentionally stated carefully. OCR, ZamanChat, and `zaman_ai` are not framed as uncontrolled autonomous systems. They are implemented as bounded support layers whose data sources, evaluation

protocols, security boundaries, and limitations are documented. This transparent framing improves the defensibility of the work.

6 CONCLUSION AND FUTURE WORK

6.1 Conclusion

This study designed and implemented Zaman Sayar, an integrated time, task, note, responsibility-management, and intelligent decision-support application. The system is built on an ASP.NET Core MVC and Entity Framework Core web architecture and is supported by SQL Server, a Python-based OCR service, a Node.js-based ZamanChat component, GitHub REST API integration, and the `zaman_ai` machine-learning subsystem. As a result, the project is not merely a set of user-interface screens; it is a multi-layered software system involving data modeling, service integration, algorithmic decisions, bounded natural-language interaction, software-development traceability, and measured decision-support outputs.

The most important result of the study is that different digital workflow needs can be addressed within a single application context. Calendar and task modules establish the relationship between work and time; notes and audio notes preserve context; the RACI structure makes responsibility sharing visible; dynamic and uploaded tables support flexible data use; statistics make records interpretable; and OCR, ZamanChat, GitHub, and Intelligent Suggestion components add auxiliary automation and decision support.

The machine-learning subsystem was evaluated on a synthetic and anonymized Turkish task dataset. It reached macro-F1 values between 0.91 and 0.93 for type, category, and priority classification; produced 1.20-hour MAE for duration estimation; reduced simulated lateness by 44.3%; and reached an 89.08% RACI acceptance rate. The summarization component achieved a 4.50/5 readability score in an AI-assisted/simulated Likert panel, which is reported transparently as a limitation rather than as a real human-subject study. These results demonstrate that the system is not only a record-keeping web application, but also a measurable and reproducible decision-support prototype.

6.2 Future Work

Future work should first strengthen authentication and authorization. User roles, module-based permissions, and ownership rules should be defined more explicitly for multi-user production scenarios. Second, the existing 55 unit tests should be expanded with controller, service, data-access, OCR parsing, GitHub integration, ZamanChat query-safety, and Intelligent Suggestion end-to-end tests.

Third, the OCR component should be evaluated with a broader set of real images and should provide clearer user feedback in failure cases. Fourth, the natural-language understanding capability of ZamanChat may be improved while preserving the strict read-only boundary. Fifth, GitHub personal access tokens should be stored in encrypted form and access scopes

should be managed more carefully. Sixth, machine-learning results should be validated on real field data, and A3 summarization should be evaluated with real human participants through a Likert-based readability study. Finally, the frontend should be further reviewed in terms of visual consistency, accessibility, mobile compatibility, and error-state experience.

In conclusion, Zaman Sayar is a comprehensive, working, explainable, measurable, and extensible software project. Its academic value does not come only from the number of features it contains, but from integrating these features around the problem of task, time, information, responsibility, software traceability, and decision support with an engineering-oriented design.

REFERENCES

- [1] Bellotti, V., Ducheneaut, N., Howard, M., Smith, I., Grinter, R. E., and Terveen, L. (2004). What a To-Do: Studies of Task Management Towards the Design of a Personal Task List Manager. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 735–742.
- [2] Payne, S. J. (1993). Understanding Calendar Use. *Human-Computer Interaction*, 8(2), 83–100.
- [3] Tungare, M., and Pérez-Quñones, M. A. (2009). An Exploratory Study of Personal Calendar Use. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 1573–1576.
- [4] Jones, W. (2007). Personal Information Management. *Annual Review of Information Science and Technology*, 41(1), 453–504.
- [5] AASHTO TAM Guide. (n.d.). Use RACI to Create a Responsibility Assignment. <https://www.tamguide.com/how-to/6-5-3-use-raci-to-create-a-responsibility-assignment/>. Accessed: 14.05.2026.
- [6] Smith, R. (2007). An Overview of the Tesseract OCR Engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition*, 629–633.
- [7] Tesseract OCR. (n.d.). Tesseract User Manual. <https://tesseract-ocr.github.io/tessdoc/>. Accessed: 14.05.2026.
- [8] JaiedAI. (2024). EasyOCR. <https://github.com/JaiedAI/EasyOCR>. Accessed: 14.05.2026.
- [9] Microsoft. (n.d.). Overview of ASP.NET Core MVC. <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview>. Accessed: 14.05.2026.
- [10] Microsoft. (n.d.). Entity Framework Core. <https://learn.microsoft.com/en-us/ef/core/>. Accessed: 14.05.2026.
- [11] Microsoft. (n.d.). SQL Server Technical Documentation. <https://learn.microsoft.com/en-us/sql/sql-server/>. Accessed: 14.05.2026.
- [12] EPPlus Software. (n.d.). EPPlus Features. <https://epplussoftware.com/en/Developers/Features>. Accessed: 14.05.2026.
- [13] Chart.js. (n.d.). Chart.js Documentation. <https://www.chartjs.org/docs/latest/>. Accessed: 14.05.2026.

- [14] MDN Web Docs. (n.d.). MediaRecorder. <https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder>. Accessed: 14.05.2026.
- [15] Express.js. (n.d.). Express Web Framework. <https://expressjs.com/>. Accessed: 14.05.2026.
- [16] Socket.IO. (n.d.). Socket.IO Documentation. <https://socket.io/docs/v4/>. Accessed: 14.05.2026.
- [17] Barreau, D., and Nardi, B. A. (1995). Finding and Reminding: File Organization from the Desktop. *ACM SIGCHI Bulletin*, 27(3), 39–43.
- [18] Whittaker, S., and Sidner, C. (1996). Email Overload: Exploring Personal Information Management of Email. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 276–283.
- [19] Chen, P. P. (1976). The Entity-Relationship Model: Toward a Unified View of Data. *ACM Transactions on Database Systems*, 1(1), 9–36.
- [20] Androutsopoulos, I., Ritchie, G. D., and Thanisch, P. (1995). Natural Language Interfaces to Databases: An Introduction. *Natural Language Engineering*, 1(1), 29–81.
- [21] Li, F., and Jagadish, H. V. (2014). Constructing an Interactive Natural Language Interface for Relational Databases. *Proceedings of the VLDB Endowment*, 8(1), 73–84.
- [22] Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.
- [23] International Organization for Standardization. (2023). ISO/IEC 25010:2023 Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model. <https://www.iso.org/standard/78176.html>. Accessed: 15.05.2026.
- [24] Salton, G., and Buckley, C. (1988). Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing & Management*, 24(5), 513–523.
- [25] Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., and Harshman, R. (1990). Indexing by Latent Semantic Analysis. *Journal of the American Society for Information Science*, 41(6), 391–407.
- [26] Mihalcea, R., and Tarau, P. (2004). TextRank: Bringing Order into Texts. *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, 404–411.

- [27] Pedregosa, F., et al. (2011). Scikit-Learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [28] Chen, T., and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- [29] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30.
- [30] Rousseeuw, P. J. (1987). Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
- [31] Caliński, T., and Harabasz, J. (1974). A Dendrite Method for Cluster Analysis. *Communications in Statistics*, 3(1), 1–27.
- [32] Davies, D. L., and Bouldin, D. W. (1979). A Cluster Separation Measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2), 224–227.
- [33] Hubert, L., and Arabie, P. (1985). Comparing Partitions. *Journal of Classification*, 2(1), 193–218.

A Appendix A: Technical Component Inventory

This appendix summarizes the technical inventory of the software components described in the main text. The aim is to show, at source-code level, which major parts constitute the implemented system.

Table 20: Technical summary of project components

Component	Description
CRM_9 web project	Contains ASP.NET Core MVC controllers, Razor views, services, static files, and main application settings.
CRM.Models	Contains entity classes for calendar, tasks, task assignments, notes, RACI, dynamic tables, uploaded tables, OCR logs, GitHub settings, and suggestion feedback.
CRM.DataAccess	Contains <code>ApplicationDbContext</code> , EF Core migrations, and database relationships.
CRM.Utility	Reserved for shared constants and helper structures.
Python/ocr_to_task.py	Performs image validation, preprocessing, Tesseract/EasyOCR extraction, field matching, and JSON result generation.
Python/zaman_ai	Provides task classification, duration estimation, summarization, RACI suggestion, clustering, and reproducible experiment reporting.
Services/OcrTaskExtractionService.cs	Manages the process contract between the web application and the Python OCR script.
Services/TaskSuggestionService.cs	Calls <code>zaman_ai.predict</code> and converts JSON output into application models.
Services/GitHubService.cs	Handles GitHub REST API communication, repositories, issues, pull requests, activity, releases, and calendar synchronization.
NodeBot/server.js	Provides the Express and Socket.IO based read-only query service for ZamanChat.
wwwroot	Contains CSS, JavaScript, chatbot client files, uploaded assets, and static interface resources.

Table 21: Suggested quality-assurance checklist

Control area	Recommended action
Build	Verify that the solution compiles successfully with the .NET 9 SDK.
Model safety	Review required/nullable fields and default values for data consistency.
Data validation	Extend validation rules for form inputs, upload limits, and JSON fields.
OCR testing	Test OCR scenarios with images of different quality and languages.
ZamanChat security	Test write-intent rejection, SQL injection resistance, and unauthorized table-access scenarios.
Machine-learning testing	Expand the existing unit tests with end-to-end suggestion-flow tests.
GitHub security	Encrypt stored personal access tokens before production use.
Interface testing	Check readability, overflow, responsiveness, and error messages on desktop and mobile screens.

B Appendix B: Source Code Excerpts

This appendix presents selected source-code excerpts from the main technical components of Zaman Sayar. The snippets are shortened for readability, but each excerpt represents a real architectural or functional structure from the implemented application.

B.1 Application Startup and Dependency Registration

Code Listing 1 shows how MVC, EF Core, the OCR service, and session management are registered during ASP.NET Core application startup. This structure is the startup-level counterpart of the layered architecture described in the paper.

```
1 var builder = WebApplication.CreateBuilder(args);
2
3 builder.Services.AddControllersWithViews();
4 builder.Services.AddScoped<IOcrTaskExtractionService, OcrTaskExtractionService>();
5
6 builder.Services.AddDbContext<ApplicationDbContext>(options =>
7     options.UseSqlServer(
8         builder.Configuration.GetConnectionString("DefaultConnection")));
9
10 builder.Services.AddDistributedMemoryCache();
11 builder.Services.AddSession();
12
13 var app = builder.Build();
14
15 app.UseHttpsRedirection();
16 app.UseStaticFiles();
17 app.UseRouting();
18 app.UseSession();
19 app.UseAuthorization();
20
21 app.MapControllerRoute(
22     name: "default",
23     pattern: "{controller=Home}/{action=Index}/{id?}");
```

Code Listing 1: ASP.NET Core application startup and service registration

B.2 Data Access Layer and Entity Relationships

Code Listing 2 shows that task, calendar, note, dynamic table, RACI, and OCR records are managed through the same database context. Relating OCR logs to task-assignment records is an important design decision for traceability.

```
1 public class ApplicationDbContext : IdentityDbContext<IdentityUser>
2 {
3     public DbSet<TaskAssignment> TaskAssignments { get; set; }
4     public DbSet<DynamicTableDefinition> DynamicTableDefinitions { get; set; }
5     public DbSet<DynamicTableData> DynamicTableData { get; set; }
6     public DbSet<CalendarTask> CalendarTasks { get; set; }
```

```

7     public DbSet<UploadedTable> UploadedTables { get; set; }
8     public DbSet<UploadedRow> UploadedRows { get; set; }
9     public DbSet<Note> Notes { get; set; }
10    public DbSet<TaskItem> TaskItems { get; set; }
11    public DbSet<DynamicBoardItem> DynamicBoardItems { get; set; }
12    public DbSet<DynamicBoardComment> DynamicBoardComments { get; set; }
13    public DbSet<OcrTaskLog> OcrTaskLogs { get; set; }
14
15    protected override void OnModelCreating(ModelBuilder modelBuilder)
16    {
17        base.OnModelCreating(modelBuilder);
18
19        modelBuilder.Entity<OcrTaskLog>()
20            .HasOne(i => i.CreatedTaskAssignment)
21            .WithMany(t => t.OcrTaskLogs)
22            .HasForeignKey(i => i.CreatedTaskAssignmentId)
23            .OnDelete(DeleteBehavior.SetNull);
24    }
25 }

```

Code Listing 2: EF Core data context and OCR relationship definition

B.3 Creating a Task Record from OCR Output

Code Listing 3 summarizes the flow in which a user-uploaded image is sent to the OCR service, extracted fields are converted into a task record, and the OCR result is stored in the log table.

```

1 public async Task<IActionResult> CreateFromImage(IFormFile imageFile)
2 {
3     var extraction =
4         await _ocrTaskExtractionService.ExtractFromImageAsync(absoluteFilePath);
5
6     var newTask = new TaskAssignment
7     {
8         TaskContent = taskContent,
9         TaskStatus = "Pending",
10        TaskDueDate = extraction.TaskDueDate ?? DateTime.Now.AddDays(1),
11        AssignedBy = assignedBy,
12        AssignedTo = assignedTo,
13        AssignedDate = DateTime.Now,
14        TaskPriority = "Medium",
15        TaskType = "OCR",
16        TaskCategory = "Automatic",
17        TaskSubCategory = extraction.Engine,
18        TaskAttachments = $"/uploads/ocr/{generatedFileName}",
19        TaskComments =
20            $"OCR engine: {extraction.Engine} | Confidence: {Math.Round(extraction.
21                Confidence, 2)}"
22    };
23
24    var ocrLog = new OcrTaskLog
25    {
26        SourceImagePath = $"/uploads/ocr/{generatedFileName}",
27        RawText = extraction.RawText ?? string.Empty,

```



```

27     Engine = string.IsNullOrEmpty(extraction.Engine)
28         ? "unknown"
29         : extraction.Engine,
30     Confidence = extraction.Confidence,
31     CreatedAt = DateTime.Now,
32     CreatedTaskAssignment = newTask
33 };
34
35 _db.TaskAssignments.Add(newTask);
36 _db.OcrTaskLogs.Add(ocrLog);
37 await _db.SaveChangesAsync();
38
39 return RedirectToAction(nameof(Index));
40 }

```

Code Listing 3: Creating task and log records from OCR output

B.4 Running the Python OCR Script from the Web Application

Code Listing 4 shows how the ASP.NET Core service runs the Python OCR script as a separate process and returns JSON output to the application layer. This approach loosely couples the .NET web application with Python-based OCR processing.

```

1 public async Task<OcrTaskExtractionResult> ExtractFromImageAsync(
2     string imagePath,
3     CancellationToken cancellationToken = default)
4 {
5     var argsBuilder = new StringBuilder();
6     argsBuilder.Append($"\"{scriptPath}\" --image \"{imagePath}\"");
7
8     var startInfo = new ProcessStartInfo
9     {
10         FileName = pythonExecutable,
11         Arguments = argsBuilder.ToString(),
12         WorkingDirectory = pythonWorkingDirectory,
13         RedirectStandardOutput = true,
14         RedirectStandardError = true,
15         UseShellExecute = false,
16         CreateNoWindow = true
17     };
18
19     using var process = Process.Start(startInfo);
20     var stdOut = await process.StandardOutput.ReadToEndAsync();
21     var stdErr = await process.StandardError.ReadToEndAsync();
22     await process.WaitForExitAsync(cancellationToken);
23
24     var parsed = TryParseJsonResult(stdOut) ?? TryParseJsonResult(stdErr);
25     if (parsed != null)
26     {
27         return parsed;
28     }
29
30     throw new InvalidOperationException("OCR result could not be read.");
31 }

```

Code Listing 4: Calling the Python OCR script from the service layer

B.5 OCR Engine Selection on the Python Side

Code Listing 5 shows that the Python script selects the better result between EasyOCR and Tesseract outputs according to a quality score. This excerpt is the code-level counterpart of the OCR scoring concept described in the paper.

```
1 def score_extraction_quality(text: str, engine: str, confidence: float) -> float:
2     fields = extract_structured_fields(text)
3     due = extract_due_date(text, fields)
4     assigned_to = extract_assigned_to(text, fields)
5     task_content = extract_task_content(text, fields)
6
7     score = 0.0
8     if due:
9         score += 0.25
10    if assigned_to:
11        score += 0.20
12    if task_content:
13        score += 0.25
14
15    score += min(max(confidence, 0.0), 1.0) * 0.30
16    return score
17
18
19 best_text, best_confidence, best_engine = max(
20     candidates,
21     key=lambda item: (
22         score_extraction_quality(item[0], item[2], item[1]),
23         score_ocr_text(item[0]),
24     ),
25 )
```

Code Listing 5: Selecting among OCR engines by quality score

B.6 Read-Only Socket.IO Flow for ZamanChat

Code Listing 6 shows how the ZamanChat service blocks write intentions, refreshes the database schema when needed, and processes only supported read/summarization requests.

```
1 io.on("connection", (socket) => {
2     socket.on("user-message", async (text, cb) => {
3         const raw = String(text || "").trim();
4         if (!raw) {
5             socket.emit("bot-response", "Empty messages are not allowed.");
6             return cb && cb({ ok: false, message: "empty_message" });
7         }
8
9         if (isWriteLike(raw)) {
```

```

10         socket.emit("bot-response", READ_ONLY_WARNING);
11         return cb && cb({ ok: false, reason: "write_forbidden" });
12     }
13
14     if (!SCHEMA.tables.length) {
15         await refreshSchema();
16     }
17
18     const action = detectAction(raw);
19     const { table, profileKey } = chooseTableAndProfile(raw);
20     if (!action || !table) {
21         socket.emit("bot-response", ACTION_HINT);
22         return cb && cb({ ok: false, reason: "unsupported_query" });
23     }
24
25     const rows = await fetchRows(table, action === "full" ? 300 : 120);
26     const answer = buildAnswer(profileKey, table, rows, action);
27     socket.emit("bot-response", answer);
28     return cb && cb({ ok: true, count: rows.length });
29 });
30 });

```

Code Listing 6: ZamanChat message handling and read-only security check

B.7 Dynamic Schema Reading Approach

Code Listing 7 shows that ZamanChat dynamically reads table and column information from SQL Server. In this way, the chatbot logic can detect database changes without depending on a fixed table list.

```

1 const tablesRes = await pool.request().query(`
2     SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_TYPE
3     FROM INFORMATION_SCHEMA.TABLES
4     WHERE TABLE_TYPE IN ('BASE TABLE', 'VIEW')
5         AND TABLE_SCHEMA NOT IN ('sys', 'INFORMATION_SCHEMA')
6     ORDER BY TABLE_SCHEMA, TABLE_NAME
7 `);
8
9 const columnsRes = await pool.request()
10     .input("schema", sql.NVarChar, table.TABLE_SCHEMA)
11     .input("table", sql.NVarChar, table.TABLE_NAME)
12     .query(`
13         SELECT c.COLUMN_NAME, c.DATA_TYPE, c.IS_NULLABLE
14         FROM INFORMATION_SCHEMA.COLUMNS c
15         WHERE c.TABLE_SCHEMA=@schema AND c.TABLE_NAME=@table
16         ORDER BY c.ORDINAL_POSITION
17     `);

```

Code Listing 7: Reading the SQL Server schema through INFORMATION SCHEMA

C Appendix C: Database Schema

This appendix summarizes the database backbone of Zaman Sayar. The schema is prepared based on the `ApplicationDbContext` class and related model classes. The aim is to show that the system is not only a collection of interfaces, but also a relational data model managing tasks, calendar entries, notes, RACI records, dynamic tables, uploaded tables, OCR logs, GitHub settings, and suggestion feedback.

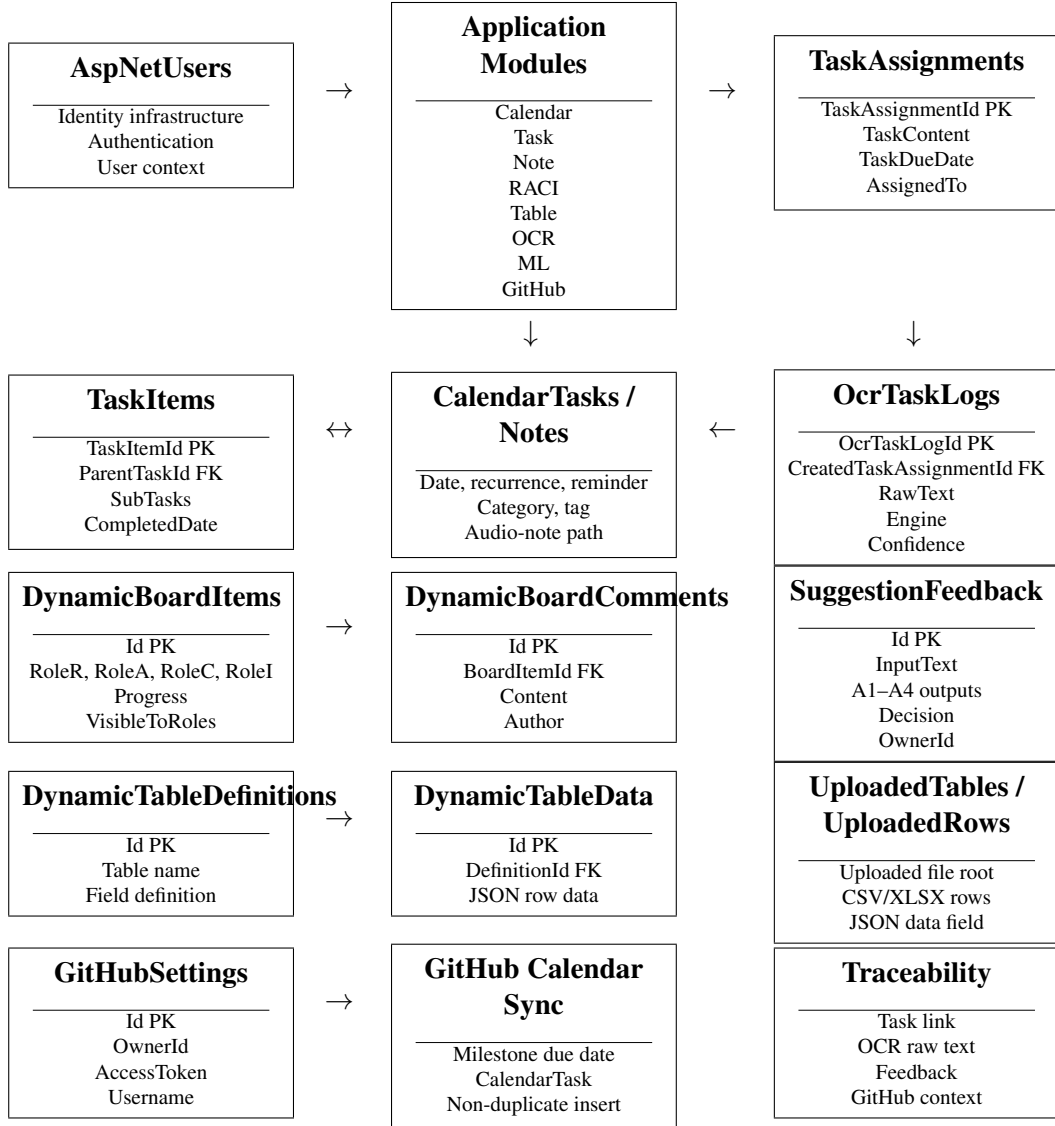


Figure 18: Zaman Sayar database schema and main relationship groups

Figure 18 shows that the database is modular but operates within a shared application context. The task-assignment module is related to OCR logs so that image-generated tasks can be audited later. In the RACI module, board records and comments are represented as separate entities. In the dynamic and uploaded table modules, row data is stored as JSON to support flexible table structures. The machine-learning feedback table stores user decisions

and model outputs, while GitHub settings connect repository information to the same owner context.

Table 22: Summary of database relationships

Source entity	Target entity	Relation	Description
TaskAssignment	OcrTaskLog	1-N	A task-assignment record may be associated with multiple OCR log records. Deletion is designed with <code>SetNull</code> .
TaskItem	TaskItem	1-N	Subtasks are represented through a parent-child relation on the same table.
DynamicBoardItem	DynamicBoardComment	1-N	Comments linked to RACI records are stored separately and removed when the main record is deleted.
DynamicTableDefinition	DynamicTableData	1-N	User-defined table structure is separated from JSON row data.
UploadedTable	UploadedRow	1-N	The uploaded Excel/CSV file is stored as a root record and rows are kept as related child records.
ApplicationUser / Owner	GitHubSetting	1-1/1-N	GitHub access settings are scoped by owner/team context.
TaskAssignment	SuggestionFeedback	0-N	ML suggestions and user decisions can be linked back to tasks where applicable.

Table 23: Main data entities and responsibilities

Entity group	Responsibility
Task and calendar entities	Manage work records, date information, recurrence, reminders, and completion states.
Note entities	Store text, category, tags, and audio-note paths within the same information-management context.
RACI entities	Make responsible, accountable, consulted, and informed roles traceable.
Dynamic table entities	Store flexible table structures and JSON row data defined by the user.
OCR entities	Preserve raw extracted text, selected OCR engine, confidence value, and the generated task link.
GitHub entities	Store GitHub account settings and connect repository/issue data to the owner context.
Suggestion feedback entities	Store model outputs, user decisions, confidence values, summaries, and acceptance-rate evidence.
Identity entities	Provide user identity and session context for the application.

CURRICULUM VITAE

PERSONAL INFORMATION

Name and Surname: Samed Alp ARSLAN

Nationality: Republic of Turkiye

E-mail: 220205012@ostimteknik.edu.tr

EDUCATION

Degree	Institution	Year
B.Sc.	OSTIM Technical University, Department of Software Engineering	2026

AREAS OF EXPERTISE

Samed Alp ARSLAN is a Software Engineering student at OSTIM Technical University. His academic and technical interests include scalable web-based software development with the .NET architecture and ASP.NET Core MVC framework, relational database design and optimization, and the integration of artificial intelligence technologies into business processes. He has worked on projects involving large language models (LLMs), generative artificial intelligence, deep learning algorithms such as CNN and RNN/LSTM, real-time data processing, and computer vision. His work particularly focuses on user-oriented information organization and task management architectures within enterprise resource planning (ERP) and customer relationship management (CRM) systems.

FOREIGN LANGUAGES

English